

RDA Microelectronics, Inc.

RDA5815S-Fully Integrated Satellite receiver for DVB-S,DVBS-S2,ABS-S,MMDS

RDA5815S Programming Guide

Confidential

Change List

Rev	Date	Author	Change Description
1.0	2011-8-31	Hongxin Wang	Original draft

Confidential

1. Introduction

RDA5815s enable software programming through I2C interface. Software controls chip working states, and allows user read status registers to get operation result through I2C interface.

2. I2C Interface

The I2C interface of RDA5815s is compliant to I2C-Bus Specification 2.1(Fast-mode, bit rate up to 400Kbit/s). It includes two pins: SCLK and SDA. SCLK is an input pin; SDA is a bi-direction pin.

The I2C interface transfer begins with START condition, a command byte and data bytes, each byte has a followed ACK (or NACK) bit, and ends with STOP condition. The command byte includes a 7-bit device address {5'b00011, MA1, MA0} and an r/w bit. (MA1 and MA0's values depends on input pin ADD's voltage, default 2'b00.) The ACK (or NACK) is always sent out by the receiver. When in write transfer process, data bytes are written out from MCU, and when in read transfer process, data bytes are read out from RDA5815s.

The RDA5815s contains status/control registers. These read/write registers are addressed as sub-address on the I2C bus. RDA5815s I2C interface supports both single and sequential register access. Software could follow the following ways to perform register read/write access:

Random access single write

Start	Device address	W	A	Register address	A	Register data	A	stop
-------	----------------	---	---	------------------	---	---------------	---	------

Random access sequential write

Start	Device address	W	A	Register N address	A	Register N Data	A	Register N+1Data	...	Register N+M Data	A	stop
-------	----------------	---	---	--------------------	---	-----------------	---	------------------	-----	-------------------	---	------

Random access single read

Start	Device address	W	A	Register address	A	start	Device address	R	A	Register Data	N	stop
-------	----------------	---	---	------------------	---	-------	----------------	---	---	---------------	---	------

Random access sequential read

Start	Device address	W	A	Register N address	A	start	Device address	R	A	Register N Data	A	Register N+1 Data	A	..	Register N+M Data	N	stop
-------	----------------	---	---	--------------------	---	-------	----------------	---	---	-----------------	---	-------------------	---	----	-------------------	---	------

W: Write Bit (0: write; 1: read)

A: Acknowledge Bit (ACK)

N: Not Acknowledge Bit (NO ACK)

For random access single write transfer, MCU sends out the START signal, RDA5815s's device address and 1 bit write signal in sequence to the I2C bus. After receiving RDA5815s' s ACK signal, MCU sends out the target register's address (8 bits) to RDA5815s and then programs this register with proper data (8 bits). A STOP signal is sent out by MCU to end this transfer when programming is finished.

For random access sequential write transfer, MCU sends out the START signal, RDA5815s's device address and 1 bit write signal to the I2C bus. After receiving RDA5815s's ACK signal, MCU sends the sequential registers' first address (8 bits) to RDA5815s. Then MCU could program these registers sequentially. A STOP signal is sent out by MCU to end this transfer when programming is finished.

For random access single read transfer, MCU first sends out the START signal, the RDA5815s's device address and 1 bit write signal to the I2C bus. After receiving RDA5815s's ACK signal, MCU sends the target register's address (8 bits) to the interface. Then MCU should send another command byte, including a RESTART signal, the RDA5815s's device address and 1 bit read signal. Then RDA5815s will send the register's data to MCU through I2C bus. After the byte has been received, MCU should send a NO ACK response signal and a STOP signal to finish this read transfer.

For random access sequential write transfer, MCU first sends out the START signal, the RDA5815s's device address and 1 bit write signal to the I2C bus. After receiving RDA5815s's ACK signal, MCU sends the sequential registers' first address (8 bits) to RDA5815s. Then MCU should send another command byte to the interface, including a RESTART signal, the RDA5815s's device address and 1 bit read signal. Then RDA5815s starts to transfer the sequential registers' data byte by byte to MCU through I2C bus, each byte followed by an ACK response signal generated by the MCU. After all target registers' data have been read, MCU should send a NO ACK signal and a STOP signal to finish this read transfer.

3. States

RDA5815s has five states: reset&initial, idle, self calibration, receive and sleep, as shown in Figure 1.

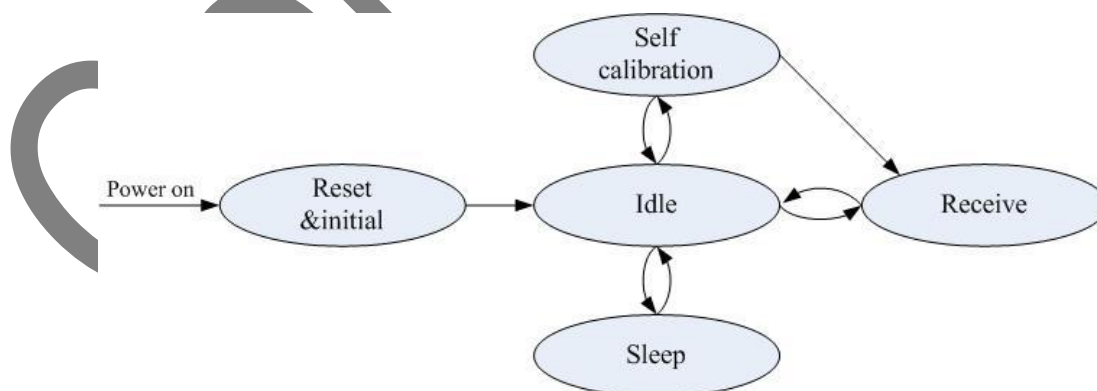


Figure 1 RDA5815s status

After chip power up, RDA5815s enters into reset&initial state automatically. Setting ENABLE (04h bit7) high and RXON bit (04h, bit1) high could bring RDA5815s enters into self calibration state to act corresponding calibration operation. If self calibration has been finished, RDA5815s will enter into receiving state (normal working state) automatically.

Software clears ENABLE bit could bring RDA5815s into sleep mode, all register contents are maintained. In sleep state, if software set ENABLE bit high, RDA5815s will enter back into normal working state.

When MCU wants to change receiving channel frequency, RXON bit should be cleared. MCU could write the frequency point value to the corresponding registers (07h, 08h, 09h and 0ah) and then set the RXON bit high.

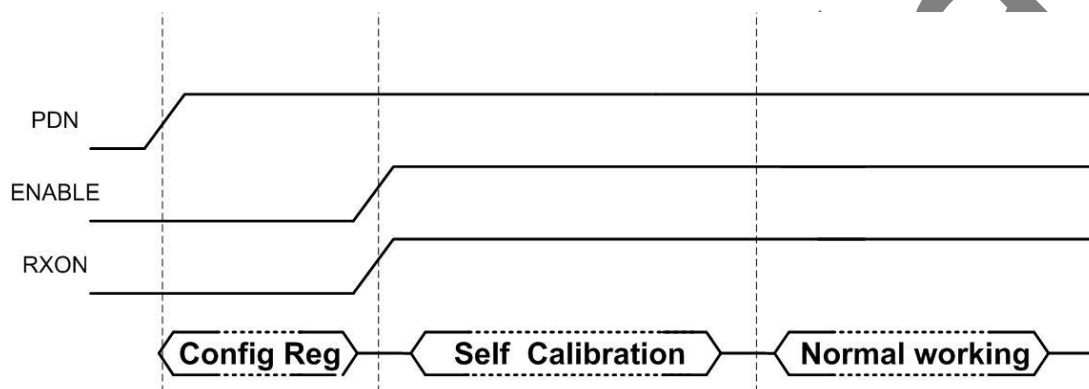


Figure 2 Normal work timing sequence

4. Frequency Setting

When frequency is changed in reset&config state, software need program correct channel frequency. The frequency value format used in RDA5815s is a 30-bit binary format, put in register 07H [5:0], 08H[7:0], 09H[7:0] and 0aH[7:0]. Software need to translate the channel frequency into 30-bit binary format, and then write into RDA5815s.

Equation of 30-bit binary format and channel frequency : (f is frequency, {07H [5:0], 08H [7:0], 09H [7:0], 0aH [7:0]} is the 30-bit format frequency value, f_xtal is the xtal frequency.)

$$\{07H[5:0], 08H[7:0], 09H[7:0], 0aH[7:0]\} = \text{dec2bin}(f * (2^{21}) / f_{\text{xtal}})$$

For example: If the channel frequency is 950MHz, xtal frequency is 27M, translate the 950MHz into 30-bit binary format:

$$\text{dec2bin}(950 * (2^{21}) / 27) = 30'b0000_0100_0110_0101_1110_1101_0000_1001$$

software should program RDA5815s register 07H,08H,09H and 0AH using the follow programming sequence:

```

Mov 8'b0000_0100(04), 07H
Mov 8'b0110_0101(65), 08H
Mov 8'b1110_1101(ed), 09H
Mov 8'b0000_1001(09), 0aH

```

Attention: Frequency changing should always be done in reset&initial state or idle state. When chip in normal working state, programmer should write 1'b0 to rxon register (04h [1]) before setting new frequency value. After frequency setting has been finished, programmer could write 1 to rxon register (04h [1]), leading the chip into self calibration state and then into receiving state again.

5. Filter bandwidth setting

The filter bandwidth of RDA5815s is adjustable (from 4M to 40M, step 1M). Software could control RDA5815s's filter bandwidth by setting register 0bh[5:0]. For example, if the target bandwidth is 10MHz, software should program 0bh [5:0] to 6'd10; if the target bandwidth is 22MHz, software should program 0bh [5:0] to 6'd22, etc...

6. Register configuration sequence

Programmer should initialize and configure RDA5815s's register file following the sequence shown in Figure 3, and a detailed register configuration example is also provided in this section.

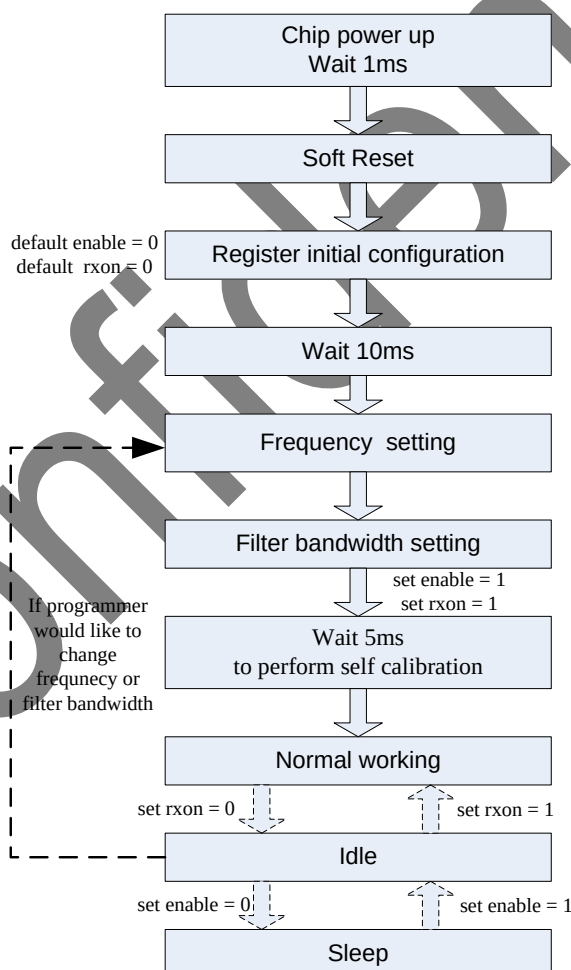


Figure 3 RDA5815 configuration sequence

Example I2C interface programming

Supply power ready.

Toggle PDN pin from low to high.

PDN pin hold high.

Wait 1ms.

```
// Chip register soft reset
```

```
RDA5815WriteReg(0x04,0x04);
```

```
RDA5815WriteReg(0x04,0x05);
```

```
//pll setting
```

```
RDA5815WriteReg(0x1a,0x13);
```

```
RDA5815WriteReg(0x38,0x90);
```

```
RDA5815WriteReg(0x39,0x15);
```

```
RDA5815WriteReg(0x3A,0x00);
```

```
RDA5815WriteReg(0x3B,0x00);
```

```
RDA5815WriteReg(0x0c,0xE2);
```

```
RDA5815WriteReg(0x2e,0x6F);
```

```
RDA5815WriteReg(0x72,0x07);
```

```
RDA5815WriteReg(0x73,0x60);
```

```
RDA5815WriteReg(0x74,0x76);
```

```
RDA5815WriteReg(0x5b,0x20);
```

```
RDA5815WriteReg(0x2f,0x57);
```

```
RDA5815WriteReg(0x0d,0x70);
```

```
RDA5815WriteReg(0x16,0x03);
```

```
RDA5815WriteReg(0x18,0x4B);
```

```
RDA5815WriteReg(0x30,0xFF);
```

```
RDA5815WriteReg(0x5c,0xFF);
```

```
RDA5815WriteReg(0x6c,0xFF);
```

```
RDA5815WriteReg(0x6e,0xFF);
```

```
RDA5815WriteReg(0x65,0x80);
```

```
RDA5815WriteReg(0x70,0x3F);
```

```
RDA5815WriteReg(0x71,0x3F);
```

```
RDA5815WriteReg(0x75,0x06);
```

```
RDA5815WriteReg(0x76,0x40);
```

```
RDA5815WriteReg(0x77,0x89);
```

```
RDA5815WriteReg(0x53,0xA8);
```

```
RDA5815WriteReg(0x46,0x21);
```

```
RDA5815WriteReg(0x47,0x84);
```

```
RDA5815WriteReg(0x48,0x10);
```

```
RDA5815WriteReg(0x49,0x08);
```

```
RDA5815WriteReg(0x60,0x80);
```

```
RDA5815WriteReg(0x61,0x80);
```

```
RDA5815WriteReg(0x6A,0x08);
RDA5815WriteReg(0x6B,0x63);
RDA5815WriteReg(0x69,0xF8);
RDA5815WriteReg(0x57,0x74);
RDA5815WriteReg(0x05,0x88);
RDA5815WriteReg(0x06,0x88);
RDA5815WriteReg(0x15,0xAA);
RDA5815WriteReg(0x4a,0xbb);
RDA5815WriteReg(0x4b,0xbb);

//agc setting
RDA5815WriteReg(0x4f,0x40);
RDA5815WriteReg(0x5b,0x20);

// for blocker
RDA5815WriteReg(0x16,0x10);//stage setting
RDA5815WriteReg(0x18,0x20);
RDA5815WriteReg(0x30,0x30);
RDA5815WriteReg(0x5c,0x30);
RDA5815WriteReg(0x6c,0x30);
RDA5815WriteReg(0x6e,0x70);
RDA5815WriteReg(0x1b,0xB2);
RDA5815WriteReg(0x1d,0xB2);
RDA5815WriteReg(0x1f,0xB2);
RDA5815WriteReg(0x21,0xB2);
RDA5815WriteReg(0x23,0xB6);
RDA5815WriteReg(0x25,0xB6);
RDA5815WriteReg(0x27,0xBA);
RDA5815WriteReg(0x29,0xBF);
RDA5815WriteReg(0xb3,0xFF);
RDA5815WriteReg(0xb5,0xFF);

RDA5815WriteReg(0x17,0xF0);
RDA5815WriteReg(0x19,0xF0);
RDA5815WriteReg(0x31,0xF0);
RDA5815WriteReg(0x5d,0xF1);
RDA5815WriteReg(0x6d,0xF2);
RDA5815WriteReg(0x6f,0xF2);
RDA5815WriteReg(0x1c,0x31);
RDA5815WriteReg(0x1e,0x72);
RDA5815WriteReg(0x20,0x96);
RDA5815WriteReg(0x22,0xBA);
```

RDA Microelectronics, Inc.**RDA5815S-Fully Integrated Satellite receiver for DVB-S,DVB-S2,ABS-S,MMDS**

```
RDA5815WriteReg(0x24,0xBA);
RDA5815WriteReg(0x26,0xBE);
RDA5815WriteReg(0x28,0xCE);
RDA5815WriteReg(0x2a,0xDE);
RDA5815WriteReg(0xb4,0x0F);
RDA5815WriteReg(0xb6,0x0F);

RDA5815WriteReg(0xb7,0x10);    //start
RDA5815WriteReg(0xb9,0x10);
RDA5815WriteReg(0xbb,0x00);
RDA5815WriteReg(0xbd,0x00);
RDA5815WriteReg(0xbf,0x00);
RDA5815WriteReg(0xc1,0x10);
RDA5815WriteReg(0xc3,0x10);
RDA5815WriteReg(0xc5,0x10);
RDA5815WriteReg(0xa3,0x19);
RDA5815WriteReg(0xa5,0x2E);
RDA5815WriteReg(0xa7,0x37);
RDA5815WriteReg(0xa9,0x47);
RDA5815WriteReg(0xab,0x47);
RDA5815WriteReg(0xad,0x3F);
RDA5815WriteReg(0xaf,0x00);
RDA5815WriteReg(0xb1,0x37);

RDA5815WriteReg(0xb8,0x47); //end
RDA5815WriteReg(0xba,0x3F);
RDA5815WriteReg(0xbc,0x37);
RDA5815WriteReg(0xbe,0x3F);
RDA5815WriteReg(0xc0,0x3F);
RDA5815WriteReg(0xc2,0x3F);
RDA5815WriteReg(0xc4,0x3F);
RDA5815WriteReg(0xc6,0x3F);
RDA5815WriteReg(0xa4,0x47);
RDA5815WriteReg(0xa6,0x57);
RDA5815WriteReg(0xa8,0x5F);
RDA5815WriteReg(0xaa,0x70);
RDA5815WriteReg(0xac,0x70);
RDA5815WriteReg(0xae,0x68);
RDA5815WriteReg(0xb0,0x37);
RDA5815WriteReg(0xb2,0x37);
```

```
RDA5815WriteReg(0x81,0x77); //rise
RDA5815WriteReg(0x82,0x68);
RDA5815WriteReg(0x83,0x70);
RDA5815WriteReg(0x84,0x68);
RDA5815WriteReg(0x85,0x68);
RDA5815WriteReg(0x86,0x68);
RDA5815WriteReg(0x87,0x70);
RDA5815WriteReg(0x88,0x47);
RDA5815WriteReg(0x89,0x68);
RDA5815WriteReg(0x8a,0x8E);
RDA5815WriteReg(0x8b,0x8E);
RDA5815WriteReg(0x8c,0x8E);
RDA5815WriteReg(0x8d,0x9C);
RDA5815WriteReg(0x8e,0x9A);
RDA5815WriteReg(0x8f,0x37);
```

```
RDA5815WriteReg(0x90,0x00); //fall
RDA5815WriteReg(0x91,0x00);
RDA5815WriteReg(0x92,0x00);
RDA5815WriteReg(0x93,0x00);
RDA5815WriteReg(0x94,0x00);
RDA5815WriteReg(0x95,0x00);
RDA5815WriteReg(0x96,0x00);
RDA5815WriteReg(0x97,0x00);
RDA5815WriteReg(0x98,0x00);
RDA5815WriteReg(0x99,0x00);
RDA5815WriteReg(0x9a,0x10);
RDA5815WriteReg(0x9b,0x24);
RDA5815WriteReg(0x9c,0x10);
RDA5815WriteReg(0x9d,0x00);
RDA5815WriteReg(0x9e,0x00);
```

Wait 10ms; // Initial configuration ready

(Attention: Initial configuration should be performed only once after chip power up or soft reset.)

```
Mov 8'b1110_0001(e1), 04H; // RXON = 0
Mov 8'b0000_0111(07), 07H; // Frequency setting (1600MHz for example)
Mov 8'b0110_1000(68), 08H;
Mov 8'b0100_1011(4b), 09H;
Mov 8'b1101_1010(da), 0aH;
Mov 8'b0000_1010(0a), 0bH; // Filter bandwidth setting (BW=10MHz for example)
```

Mov 8'b1110_0011(e3), 04H; // Pull up enable and rxon ; AGC enable;

Wait 5ms;

// Normal working start ... (Configuration all ready)

.....

// When programmer would like to change working frequency to 1700MHz and filter band width to 5M, the following command sequence should be performed:

Mov 8'b1110_0001(e1), 04H; // RXON = 0 , change normal working state to idle state

Mov 8'b0000_0111(07), 07H; // Frequency setting (1700MH)

Mov 8'b1101_1110(de), 08H;

Mov 8'b1101_0000(d0), 09H;

Mov 8'b1001_1000(98), 0aH;

Mov 8'b0000_0101(05), 0bH; // Filter bandwidth setting (change BW to 5MHz)

Mov 8'b1110_0011(e3), 04H; // Pull up enable and rxon ; AGC enable;

Wait 5ms;

// Enter normal working state again ... (Configuration all ready)

7. Register summary

Register 00h Chip ID_1 (read only) default: 8'b0101_1000

Bit	Name	Function	Note
7:0	Chip_ID[11:4]	5812 chip id Default 8'b0101_1000	read only

Register 01h Chip_ID_2 (read only) default: 8'b1100_0000

Bit	Name	Function	Note
7:4	Chip_ID[3:0]	Default: 4'b1100	read only
3:0	Revision_ID	Default: 4'b0000	read only

Register 02h Device ID (read only) default: 8'b0000_1100

Bit	Name	Function	Note
7	Reversed	1'b0	
6:2	Device_ID_fix[4:0]	Chip address Fixed Default 5'h0_0011	read only
1	MA1	address 1 selected via pin ADD configuration	read only

RDA Microelectronics, Inc.

RDA5815S-Fully Integrated Satellite receiver for DVB-S,DVB-S2,ABS-S,MMDS

		Default 0	
0	MA0	address 1 selected via pin ADD configuration Default 0	read only

Register 04h Config Control register

default: 8'b0000_0101

Bit	Name	Function	Note
7	enable	if 1,enable interface; if 0,sleep mode, only bandgap,xtal and adr_encoder module power on Default: 1'b0	
6	Bypass_rxon_agc_enable	Default: 1'b0	
5	rf_bypass_enable_b	0: power up 1: power down Default: 1'b0 The bypass buf functions at power-up of the chip by default. If this function is not required, set this bit to logic '1', which will save about 6mA current.	
4	bypass_tuning_enable	If set to 1,bypass tuning calibration when changing the filter bandwidth; Default: 1'b0	
3	bypass_rxon_dc_cal_enable	If set to 1,bypass dc offset calibration when each time changing the working freq or the filter bandwidth; Default: 1'b0	
2	agc_en_b	If 0,AGC enable; If 1,disable AGC; Default 1	
1	rxon	If 1 , in rxon mode Default 0	
0	Soft_resetrn	If 0, then reset all the register of chip Default 1	

Register 07h PLL frequency setting1 (950MHZ)

default: 8'b0000_0100

Bit	Name	Function	Note
7:0	Freq_synthesize_reg [31:24]	pll freq setting , from MSB to LSB :10 integer bits,22 decimal [7:2] integer bits	

RDA Microelectronics, Inc.**RDA5815S-Fully Integrated Satellite receiver for DVB-S,DVB-S2,ABS-S,MMDS**

Register 08h PLL frequency setting 2 default: 8'b0110_0101

Bit	Name	Function	Note
7:6	Freq_synthesize_reg [23:22]	[1:0] integer bits	
5:0	Freq_synthesize_reg [21:16]	pll freq setting , 22 decimal bits [21:16]. decimal bits[21:16].	

Register 09h PLL frequency setting 3 default:8'b1110_1101

Bit	Name	Function	Note
7:0	Freq_synthesize_reg [15:8]	pll freq setting , 22 decimal bits[15:8]. decimal bits[15:8].	

Register 0ah PLL frequency setting 4 default:8'b0000_1001

Bit	Name	Function	Note
7:0	Freq_synthesize_reg [7:0]	pll freq setting , 22 decimal bits[7:0]. decimal bits[7:0].	

Register 0bh filter bandwidth setting register default:8'b0000_1010

Bit	Name	Function	Note
7	filter_bw_control_bit_extra	Default 1'b0 when the filter bandwidth between 4M~8M, we use this bit to make the BW precision to 0.5M. eg: when BW=4M,we could set filter_bw_control_bit 6'b00_0100,and filter_bw_control_bit_add 1'b0; when BW 4.5M,we could set filter_bw_control_bit 6'b00_0100,and filter_bw_control_bit_add 1'b1;	
6	filter_fc_bit_dr	If 1,filter fc control bits using value direct decoded from filter_bw_control_bit; If 0, filter fc control bits using tuning calibration result. default: 0	
5:0	filter_bw_control_bit [5:0]	filter bandwidth setting(4MHz ~ 40MHz) eg;if user need bandwidth 10M, set this register to 6'b00_1010.	

APPENDIX1 Different crystal application

The rda5815s support the different frequency crystal According to the different frequency crystal, the register 0x72~0x77 and 0x79~0x7c should be set to the different values.

1, The register 0x72~0x74 should be set as follows.

a> $\text{data} = \text{dec2bin}(1600 * (2^{21}) / \text{crystal})$. Data should be take 32bit[0~31bit] and the high bit be set to 0.

b> The register 0x72 should be set to the value of data's 31~24bit. The 7~4bit of register 0x73 should be set to the value of data's 23~20bit. The 3~0bit of register 0x73 should be set to the value of data's 31~28bit. The register 0x74 should be set to the value of data's 27~20bit.

For example, the frequency of crystal is equal to 10M.

$\text{data} = \text{dec2bin}(1600 * (2^{21}) / 10) = 0001\ 0100\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$

Mov 8'b0001_0100, 72H Mov 8'b0000_0001, 73H Mov 8'b0100_0000, 74H

For example, the frequency of crystal is equal to 27M.

$\text{data} = \text{dec2bin}(1600 * (2^{21}) / 10) = 0000\ 0111\ 0110\ 1000\ 0100\ 1011\ 1101\ 1010$

Mov 8'b0000_0111, 72H Mov 8'b0110_0000, 73H Mov 8'b0111_0110, 74H

2, The register 0x79~0x7c should be set as follows.

a> $\text{data} = \text{dec2bin}(1075 * (2^{21}) / \text{crystal})$. Data should be take 32bit[0~31bit] and the high bit be set to 0.

b> The value of the register from 0x79 to 0x7c should be set according to value of data from high bit to low bit.

For example, the frequency of crystal is equal to 10M.

$\text{data} = \text{dec2bin}(1075 * (2^{21}) / 10) = 0000\ 1101\ 0111\ 0000\ 0000\ 0000\ 0000\ 0000$

Mov 8'b0000_1101, 79H Mov 8'b0111_0000, 7aH Mov 8'b0000_0000, 7bH Mov 8'b0000_0000, 7cH

For example, the frequency of crystal is equal to 27M.

$\text{data} = \text{dec2bin}(1075 * (2^{21}) / 27) = 0000\ 0100\ 1111\ 1010\ 0001\ 0010\ 1111\ 0110$

Mov 8'b0000_0100, 79H Mov 8'b1111_1010, 7aH Mov 8'b0001_0010, 7bH Mov 8'b1111_0110, 7cH

3, The register 0x75~0x77 should be set as follows.

a> $\text{data1} = \text{dec2bin}(1350 * (2^{21}) / \text{crystal})$. Data should be take 32bit[0~31bit] and the high bit be set to 0.

$\text{data2} = \text{dec2bin}(1850 * (2^{21}) / \text{crystal})$. Data should be take 32bit[0~31bit] and the high bit be set to 0.

b> The register 0x75 should be set to the value of data1's 31~24bit. The 7~4bit of register 0x76 should be set to the value of data1's 23~20bit. The 3~0bit of register 0x76 should be set to the value of data2's 31~28bit. The register 0x77 should be set to the value of data2's 27~20bit.

For example,the frequency of crystal is equal to 10M.

$\text{data1} = \text{dec2bin}(1350 * (2^{21}) / 10) = 0001\ 0000\ 1110\ 0000\ 0000\ 0000\ 0000\ 0000$

$\text{data2} = \text{dec2bin}(1850 * (2^{21}) / 10) = 0001\ 0111\ 0010\ 0000\ 0000\ 0000\ 0000\ 0000$

Mov 8'b0001_0000, 75H Mov 8'b1110_0001, 76H Mov 8'b0111_0010, 77H

For example,the frequency of crystal is equal to 27M.

$\text{data1} = \text{dec2bin}(1350 * (2^{21}) / 27) = 0000\ 0110\ 0100\ 0000\ 0000\ 0000\ 0000\ 0000$

$\text{data2} = \text{dec2bin}(1850 * (2^{21}) / 27) = 0000\ 1000\ 1001\ 0000\ 1001\ 0111\ 1011\ 0100$

Mov 8'b0000_0110, 75H Mov 8'b0100_0000, 76H Mov 8'b1000_1001, 77H

RDA Microelectronics, Inc.

RDA5815S-Fully Integrated Satellite receiver for DVB-S,DVB-S2,ABS-S,MMDS

Disclaimer

The information provided here is believed to be reliable; RDA Microelectronics assumes no reliability for inaccuracies and omissions. RDA Microelectronics assumes no reliability for the use of this information and all such information should entirely be at the user's own risk. Specifications described and contained here are subjected to change without notice on the purpose of improving the design and performance. All of this information described herein should not be implied or granted for any third party. RDA Microelectronics does not authorize or warrant any RDA products for use in the life support devices or systems.

Copyright©2006 RDA Microelectronics Inc. All rights reserved

For technical questions and additional information about RDA Microelectronics Inc.:

Website: www.rdamicro.com

Mailbox: info@rdamicro.com

RDA Microelectronics (Shanghai), Inc.

Tel: +86-21-50271098

Fax: +86-21-50271099

RDA Microelectronics (Beijing), Inc.

Tel: +86-10-62635360

Fax: +86-10-82612663